

Spring MVC와 Thymeleaf

학습 체크리스트 (1/2)

- ☐ DispatcherServlet 중심의 Spring MVC 요청 처리 흐름 이해
- ☐ ViewResolver의 논리 뷰 이름 → 템플릿 경로 해석 방식 이해
- ☐ forward와 redirect의 차이, PRG(Post-Redirect-Get) 패턴 이해
- ☐ `@PathVariable` · `@RequestParam` · `@ModelAttribute` 바인딩 방식 습득

학습 체크리스트 (2/2)

☐

`Model` · `RedirectAttributes` 로 데이터와 일회성 메시지 전달

☐

`th:object` · `th:field` 로 폼을 커맨드 객체에 바인딩하는 방법 습득

☐

CRUD 라우팅 설계(GET/POST 조합)와 화면 구성 습득

☐

폼 전용 DTO 사용 등 안전한 바인딩 원칙 이해

Spring MVC 동작 구조 이해하기

모든 요청이 거치는 첫 관문: DispatcherServlet

프론트 컨트롤러 패턴 (Front Controller Pattern)

모든 요청을 하나의 공통 진입점이 먼저 받아 실제 처리를 위임하는 설계 패턴

- Spring MVC는 URL마다 별도 서블릿을 두지 않고 진입점을 하나로 통일
- `DispatcherServlet`이 그 진입점이며, 실제 처리는 내부 컴포넌트에 위임
- 요청 처리 흐름을 공통 진입점에서 일관되게 조정할 수 있는 구조

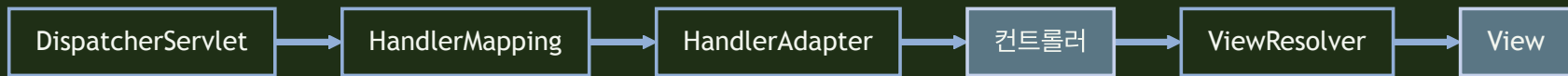
스프링 부트의 DispatcherServlet 자동 설정

- Spring Boot 4에서는 `spring-boot-starter-webmvc`로 Spring MVC를 구성
- Spring Boot 3 이하 자료에서는 주로 `spring-boot-starter-web`을 사용
- 별도 설정 없이 `/` 경로 전체에 매핑됨
- `web.xml`이나 `WebApplicationInitializer`를 직접 작성할 필요 없음

서버 렌더링 요청이 처리되는 대표 흐름

단계	담당	역할
1	DispatcherServlet	모든 요청을 최초로 수신
2	HandlerMapping	URL에 맞는 컨트롤러(핸들러) 탐색
3	HandlerAdapter	찾은 핸들러를 실제로 실행
4	컨트롤러	요청 해석·서비스 호출 후 뷰 이름 반환
5	ViewResolver	논리 뷰 이름을 실제 View로 해석
6	View	데이터를 담아 HTML로 렌더링

한눈에 보는 MVC 요청 처리 흐름



우리가 구현할 핵심 영역: 오직 컨트롤러

- 6단계 중 직접 작성하는 부분은 4단계(컨트롤러) 하나뿐
- 요청 수신·핸들러 탐색·실행·뷰 해석·렌더링은 프레임워크가 처리
- 렌더링 결과는 다시 DispatcherServlet을 거쳐 브라우저로 응답됨
- 컨트롤러는 서비스 호출과 뷰 이름 반환에만 집중

논리 뷰 이름을 실제 경로로 매핑하는 ViewResolver

- 컨트롤러가 반환한 논리 뷰 이름을 실제 템플릿 경로로 변환
- 기본 prefix는 `classpath:/templates/`, 기본 suffix는 `.html`
- 변환 규칙은 `prefix + 논리 이름 + suffix` 형태로 조합됨
- 논리 이름의 슬래시(`/`)는 실제 디렉터리 구조에 그대로 대응

뷰 이름 매핑의 실제 동작 방식

구분	값
컨트롤러 반환값	<code>"books/list"</code>
prefix	<code>classpath:/templates/</code>
suffix	<code>.html</code>
실제 경로	<code>classpath:/templates/books/list.html</code>

템플릿 경로와 개발 편의를 위한 캐시 설정

```
1  spring:
2    thymeleaf:
3      prefix: classpath:/templates/
4      suffix: .html
5      cache: false
```

화면을 전환하는 두 가지 방법: Forward와 Redirect

- 일반 뷰 이름을 반환하면 View가 곧바로 렌더링됨
- `"forward:/other"` 는 서버 내부에서 다른 경로로 흐름만 이관
- `"redirect:/books"` 는 302 응답을 보내 브라우저가 새로 요청하도록 유도
- 두 방식 모두 문자열 접두사만으로 동작이 결정됨

Forward와 Redirect 동작 차이 비교

구분	요청 횟수	URL 변화	데이터 전달
forward	1회	유지됨	request 속성 공유
redirect	2회	변경됨	request 속성 소멸

- redirect는 새 요청이므로 이전 request에 담긴 값을 이어받지 못함

뷰 이름 접두사로 리다이렉트 지시하기

```
1 return "redirect:/books"; // 브라우저가 /books로 재요청
```

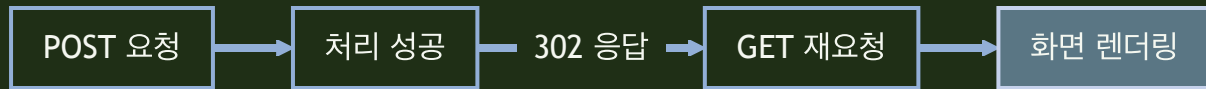
새로고침으로 발생하는 중복 등록 문제

PRG 패턴 (Post-Redirect-Get Pattern)

POST 처리 후 뷰를 직접 렌더링하지 않고 GET으로 리다이렉트해 재전송을 막는 기법

- POST 처리 후 뷰를 바로 렌더링하면 새로고침 시 같은 POST가 재전송됨
- 재전송된 POST가 등록·수정·삭제를 다시 실행해 중복 처리를 유발
- POST 처리 뒤 GET 요청으로 유도해 이 문제를 차단

중복 등록을 원천 차단하는 PRG 패턴 흐름



상황별 응답 분기: 성공(Redirect)과 실패(Render)

- 등록·수정·삭제가 **성공**하면 redirect로 응답해 이후 GET이 화면을 그림
- **검증 실패** 시에는 redirect 없이 폼 뷰를 직접 다시 렌더링
- 실패 케이스에서 redirect를 쓰면 입력값과 오류 메시지가 사라짐

입력 정보의 증발 (Data Loss)

입력값 검증에 실패했을 때 리다이렉트(Redirect)를 쓰면 브라우저 요청이 완전히 새로 생성되어 사용자가 입력한 값과 오류 정보가 소멸합니다.

등록 완료 시의 Redirect 반환

```
1  @PostMapping("/books")
2  String create(@ModelAttribute("bookForm") BookForm form) {
3      bookService.create(form);
4      return "redirect:/books"; // 성공 시 GET으로 재요청 유도
5  }
```

컨트롤러 구현 및 요청 데이터 바인딩

요청 매핑을 처리하는 어노테이션과 HTTP 메서드

- 클래스 레벨 `@RequestMapping` 으로 공통 경로를 지정
- 메서드 레벨 `@GetMapping` · `@PostMapping` 으로 세부 경로와 HTTP 메서드를 지정
- HTML `<form>` 은 GET/POST만 지원 → 수정·삭제도 대부분 POST로 처리
- `HiddenHttpMethodFilter` 가 `_method` 를 PUT/DELETE로 바꿔주지만 기본 비활성화

도서 관리 컨트롤러의 기본 설계 골격

```
1  @Controller
2  @RequestMapping("/books")
3  class BookController {
4      @GetMapping("/{id}")
5      String detail(@PathVariable("id") Long id) { ... }
6
7      @PostMapping("/{id}/delete")
8      String delete(@PathVariable("id") Long id) { ... }
9  }
```

경로의 값을 변수로 추출하는 @PathVariable

- 경로에 포함된 값을 리소스 식별자로 바인딩
- `-parameters` 컴파일 옵션이 보장되지 않는 환경까지 고려해 이름 명시가 안전
- `@PathVariable("id")` 형태로 매핑할 이름을 직접 선언

경로 변수를 이용한 식별자 매핑

```
1 @GetMapping("/{id}")
2 String detail(@PathVariable("id") Long id, Model model) {
3     model.addAttribute("book", bookService.findById(id));
4     return "books/detail";
5 }
```


쿼리 파라미터를 받는 @RequestParam

- 요청 파라미터(쿼리 스트링 또는 폼 데이터)를 핸들러 메서드의 매개변수에 매핑
- 어노테이션에 대상 파라미터 이름을 명시하는 것이 컴파일 최적화 환경에서 가장 안전
- 기본적으로 선언된 파라미터는 필수값(`required = true`)으로 취급됨

@RequestParam의 세부 속성 활용

- `value` / `name`: 바인딩할 요청 파라미터의 이름을 명시
- `required`: 필수 여부 설정 (`false` 지정 시 해당 파라미터가 없어도 에러 미발생)
- `defaultValue`: 파라미터가 비어있을 때 적용할 기본값을 설정 (설정 시 `required=false` 효과 자동 부여)

선택적 파라미터 처리 (Optional)

`Optional<String>` 처럼 선택적 값을 표현하는 컨테이너 타입으로 파라미터 누락을 처리할 수도 있습니다.
이 경우 `required=false` 와 같은 의미로 동작합니다.

요청 파라미터 바인딩 적용

```
1  @GetMapping
2  String list(@RequestParam(value = "keyword", defaultValue = "") String keyword,
3              Model model) {
4      model.addAttribute("books", bookService.search(keyword));
5      return "books/list";
6  }
```

요청 데이터를 객체로 일괄 수신하는 `@ModelAttribute`

- 요청 파라미터를 커맨드 객체 필드에 한 번에 바인딩
- 바인딩된 객체는 자동으로 Model에 담겨 뷰에서 바로 접근 가능
- 속성 이름 기본값은 클래스명 첫 글자를 소문자로 바꾼 형태
- 메서드 레벨 `@ModelAttribute` 는 해당 컨트롤러의 핸들러 실행 전에 공통 데이터를 채움

모든 요청 모델에 공통 데이터 채우기

```
1 @ModelAttribute("categories")
2 List<Category> categories() {
3     return categoryService.findAll();
4 }
```

데이터를 전달하는 Model과 뷰를 묶는 ModelAndView

- 메서드 매개변수로 `Model` 을 선언하면 자동 주입됨
- `addAttribute` 로 뷰에 전달할 데이터를 담음
- `ModelAndView` 는 모델 데이터와 뷰 이름을 한 객체에 함께 담는 방식
- 일반 CRUD에서는 `Model` + 문자열 반환 조합이 더 일반적

여러 요청에 걸쳐 상태를 유지하는 HttpSession

- 메서드 매개변수로 `HttpSession` 을 선언하면 스프링이 이를 자동 주입
- 로그인한 사용자 정보 등 여러 페이지 요청에 걸쳐 지속되어야 하는 유저 상태 관리
- 전체 세션 객체가 아닌 특정 데이터만 꺼내고자 할 때는 `@SessionAttribute` 활용

특정 세션 정보만 집중 수신하는 @SessionAttribute

- 세션 정밀 타겟팅: 세션에서 특정 key에 해당하는 객체만 매개변수로 자동 매핑
- 주요 속성: `name` (조회할 세션 속성 이름 명시), `required` (해당 속성의 필수 여부 설정)
- 보안 정보 관리: 직접 서블릿 API(`request.getSession()`)를 조작할 필요가 없어 코드가 간결해짐

세션 사용자 정보 직접 조회

```
1  @GetMapping("/mypage")
2  String myPage(
3      @SessionAttribute(name = "loginMember", required = false) Member member,
4      Model model
5  ) {
6      if (member == null) return "redirect:/login";
7      model.addAttribute("profile", member);
8      return "members/mypage";
9  }
```

리다이렉트 시 안전한 정보 전달: RedirectAttributes

메서드	저장 위치	URL 노출
<code>addAttribute</code>	URL 쿼리 스트링·경로 변수	노출됨
<code>addFlashAttribute</code>	세션(1회성)	노출 안 됨

- flash 속성은 리다이렉트된 요청에서 한 번만 꺼내지고 사라짐

화면에 일회성 메시지를 뿌리는 Flash Attribute

```
1 @PostMapping
2 String create(@ModelAttribute("bookForm") BookForm form, RedirectAttributes ra) {
3     bookService.create(form);
4     ra.addFlashAttribute("message", "등록되었습니다.");
5     return "redirect:/books";
6 }
```

메시지 미표출 오류 (Muted Message)

일반 `Model` 은 리다이렉트되는 순간 소멸합니다. 리다이렉트 이후 화면까지 알림 메시지를 운반하려면 `RedirectAttributes` 의 `addFlashAttribute` 를 써야 합니다.

엔티티 보호와 보안을 위한 폼 전용 DTO

대량 할당 (Mass Assignment)

요청 파라미터가 객체 필드에 그대로 바인딩되어 의도하지 않은 값까지 함께 변경되는 문제

- 요청 파라미터를 영속 엔티티에 직접 바인딩하면 대량 할당 위험이 생김
- 웹 입력 전용 `BookForm` DTO를 쓰고 서비스에서 엔티티로 변환
- 엔티티 구조 변경이 폼 요청 형식에 직접 노출되지 않음

대량 할당 보안 취약점 (Mass Assignment)

영속성 엔티티에 파라미터를 직접 바인딩하면 악의적인 수정 폼 조작(예: 관리자 필드 임의 주입 등)에 취약해지므로 폼 DTO 분리가 필수입니다.

서블릿과 Spring MVC 개발 방식 비교 (1/2)

기능	Servlet	Spring MVC
파라미터 조회	<code>request.getParameter(...)</code> 후 직접 형변환	<code>@RequestParam</code> 이 선언 타입으로 자동 변환
객체 바인딩	파라미터를 하나씩 꺼내 setter 호출	<code>@ModelAttribute</code> 가 일괄 바인딩
뷰 전달	<code>request.setAttribute(...)</code>	<code>Model.addAttribute(...)</code>

서블릿과 Spring MVC 개발 방식 비교 (2/2)

기능	Servlet	Spring MVC
화면 이동	<code>RequestDispatcher.forward</code> / <code>sendRedirect</code>	뷰 이름의 <code>forward:</code> · <code>redirect:</code> 접두사
세션 사용	<code>request.getSession()</code> 호출	매개변수로 <code>HttpSession</code> 선언
반환 타입	<code>void</code> , 응답을 직접 작성	뷰 이름 문자열 반환

타임리프 폼 바인딩과 th:field

폼 태그에 커맨드 객체를 바인딩하는 th:object

선택 변수 표현식 (Selection Variable Expression)

`th:object`로 지정된 객체를 기준으로 프로퍼티를 참조하는 `*{...}` 표현식

- `th:object`는 폼 전체를 하나의 커맨드 객체에 연결
- 폼 내부에서는 `*{...}`로 그 객체의 프로퍼티에 접근
- `th:field`를 쓰려면 `<form th:object="${ ... }">` 지정이 먼저 필요

폼 객체 누락에 따른 바인딩 오류 (Binding Error)

Spring 폼에서는 `th:object`와 `th:field="*{ ... }"`를 함께 사용합니다. 선택 객체나 폼 객체가 없으면 템플릿 처리 중 표현식 평가 또는 바인딩 오류가 발생할 수 있습니다.

등록 폼에 th:object와 th:field 적용

```
1 <form th:action="@{/books}" th:object="${bookForm}" method="post">
2   <input type="text" th:field="*{title}" />
3   <button type="submit">등록</button>
4 </form>
```

th:action의 경로 계산과 CSRF 토큰 처리

- `th:action="@{ ... }"` 은 애플리케이션의 컨텍스트 경로를 반영해 URL을 생성
- Spring Security가 연동된 POST 폼에는 CSRF 토큰이 hidden input으로 자동 추가
- CSRF 보호가 활성화된 변경 요청은 유효한 토큰이 있어야 처리됨

th:field가 자동 생성하는 HTML 속성들

생성 속성	값의 출처
<code>id</code>	프로퍼티 경로
<code>name</code>	프로퍼티 경로
<code>value</code>	커맨드 객체의 현재 값
<code>checked</code> / <code>selected</code>	값과의 일치 여부

- `th:field="*{title}"` 한 줄로 위 속성들이 함께 생성됨

선택형 입력 요소(체크박스)의 상태 자동 매핑

```
1 <input type="checkbox" th:field="*{available}" />
```

- 체크박스·선택트박스는 값에 맞춰 `checked` · `selected` 까지 처리됨

폼 화면을 열기 위한 컨트롤러의 기본 준비

- 폼 진입 컨트롤러는 빈 커맨드 객체를 `Model`에 담아야 `th:object`가 동작
- 객체가 없으면 `th:field`가 참조할 프로퍼티 경로 자체가 사라짐
- 등록 폼과 수정 폼 모두 같은 규칙이 적용됨

컨트롤러에서 빈 커맨드 객체 제공하기

```
1 @GetMapping("/books/new")
2 String newForm(Model model) {
3     model.addAttribute("bookForm", new BookForm("", 0));
4     return "books/form";
5 }
```

불변 record 객체를 폼 DTO로 활용하기

- `record` 는 생성자 바인딩으로 요청값을 받는 폼 DTO로 사용 가능
- 생성자 매개변수로 허용할 입력 필드가 명확해 안전한 바인딩에 유리
- 중첩 객체와 컬렉션도 생성자 매개변수로 바인딩 가능

폼의 변경 방식에 따른 DTO 선택

- 요청 한 번으로 완성되는 입력 모델은 불변 `record`가 간결하고 안전함
- 기존 객체를 단계적으로 변경해야 하면 setter를 가진 전용 DTO가 편리할 수 있음
- 동적 행 추가·삭제는 DTO의 불변성보다 폼 필드 구조와 컬렉션 설계가 중요
- 어떤 방식을 쓰더라도 영속 엔티티 대신 입력 전용 DTO로 바인딩 범위를 제한

CRUD 화면 구현

CRUD 구현을 위한 라우팅 설계: 조회 (GET)

동작	메서드	경로
목록	GET	/books
상세	GET	/books/{id}
등록 폼	GET	/books/new
수정 폼	GET	/books/{id}/edit

CRUD 구현을 위한 라우팅 설계: 변경 (POST)

동작	메서드	경로
등록 처리	POST	<code>/books</code>
수정 처리	POST	<code>/books/{id}/edit</code>
삭제 처리	POST	<code>/books/{id}/delete</code>

- 변경 동작은 HTML 폼 제약 때문에 모두 POST로 통일

목록 화면에서 상세 화면 연결 설계

- 목록은 `th:each` 로 행을 반복해 데이터를 표시
- 각 행에서 상세 화면으로 이동하는 링크를 생성
- 링크 URL은 `@{...}` 에 경로 변수를 넘겨 조립

th:each와 경로 변수를 활용한 상세 링크 생성

```
1 <tr th:each="book : ${books}">
2   <td th:text="${book.title}">제목</td>
3   <td><a th:href="@{/books/{id}(id=${book.id})}">상세</a></td>
4 </tr>
```

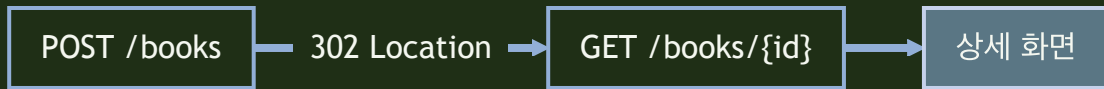
등록 완료 후 상세 화면으로 Redirect

- POST 처리 후 뷰 이름을 바로 반환하지 않고 저장된 리소스 상세로 리다이렉트
- 새로고침해도 폼이 재전송되지 않아 중복 등록이 방지됨
- 성공 메시지는 flash 속성으로 다음 화면에 전달

등록 컨트롤러에서 Redirect 응답 반환

```
1 @PostMapping("/books")
2 String create(BookForm form, RedirectAttributes ra) {
3     Book saved = bookService.create(form);
4     ra.addFlashAttribute("message", "등록되었습니다");
5     return "redirect:/books/%d".formatted(saved.getId());
6 }
```

등록 요청의 전체 왕복 흐름



기존 데이터를 불러와 처리하는 수정 흐름

- 수정 폼은 기존 값을 담은 커맨드 객체를 `Model`에 넣어 `th:field`가 값을 채우게 함
- 처리 후에는 등록과 동일하게 리다이렉트
- 등록 폼과 수정 폼은 같은 템플릿을 재사용할 수 있음

수정용 컨트롤러에서 기존 데이터 채워주기

```
1 @GetMapping("/books/{id}/edit")
2 String editForm(@PathVariable("id") Long id, Model model) {
3     model.addAttribute("bookForm", bookService.findFormById(id));
4     return "books/form";
5 }
```

리소스 삭제 시 반드시 POST 메서드 사용

- HTML 폼은 GET/POST만 보낼 수 있어 DELETE를 직접 전송할 수 없음
- 삭제 버튼은 링크가 아닌 별도의 POST 폼으로 구성
- 링크(GET)로 삭제를 처리하면 크롤러나 프리페치에 의해 실행될 위험이 있음

타임리프 기반의 삭제 전용 POST 폼 구현

```
1 <form th:action="@{/books/{id}/delete(id=${book.id})}" method="post">
2   <button type="submit">삭제</button>
3 </form>
```

화면에 처리 완료 메시지 보여주기

```
1 <p th:if="${message}" th:text="${message}">알림</p>
```

- 리다이렉트 후에도 살아남으려면 `addFlashAttribute` 로 담아야 함

개발 편의성과 생산성을 높이는 부가 설정

```
1  spring:
2    thymeleaf:
3      cache: false
4    devtools:
5      restart:
6        enabled: true
```

- `spring-boot-devtools` 의존성이 있어야 재시작 설정이 적용됨
- 템플릿 캐시 비활성화는 브라우저에서 다시 요청할 때 변경 내용을 반영
- 클래스패스 파일 변경 시 DevTools가 애플리케이션을 자동 재시작

학습 요약 (1/2)

- Spring MVC 동작 구조:
 - DispatcherServlet이 요청을 받아 HandlerMapping·ViewResolver를 거쳐 화면을 완성하는 흐름 이해
- forward·redirect와 PRG 패턴:
 - POST 처리 후 redirect로 이동해 새로고침 중복 등록을 방지하는 원리 이해
- 컨트롤러와 요청 바인딩:
 - `@PathVariable` · `@RequestParam` · `@ModelAttribute` 로 요청 데이터를 수신하는 방법 습득

학습 요약 (2/2)

- 폼 바인딩과 th:field:
 - `th:object` 로 지정한 커맨드 객체에 `th:field` 가 값과 속성을 자동 연결하는 원리 습득
- CRUD 화면 구현:
 - 목록·상세·등록·수정·삭제에 필요한 GET/POST 라우팅과 뷰 구성 설계 습득
- 안전한 바인딩 원칙:
 - 폼 전용 DTO와 flash 속성, CSRF 보호를 유지하는 화면 설계 이해